GENERALITAT VALENCIANA
Conselleria d'Educació, Universitats i Ocupació

ceedcv CENTRE ESPECÍFIC D'EDUCACIÓ A DISTÀNCIA DE LA COMUNITAT VALENCIANA

Unidad 02 — Gestión de procesos
Bloque 01

2º CFGS DAM · Programación de Servicios y Procesos

Sergi García Barea

Qué es un proceso

La idea en una frase

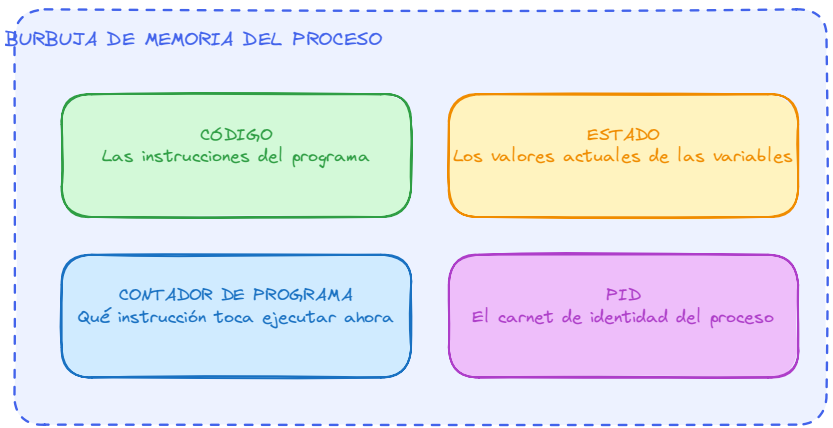
> Un **proceso** es un programa en ejecución con su propio espacio de memoria, recursos (archivos abiertos, sockets) y un identificador único llamado **PID**.

Un programa en el disco duro es un **muerto viviente**: no hace nada. Un proceso es ese mismo programa **vivo**, ocupando memoria, consumiendo CPU y respondiendo al teclado, a la red o al ratón.

La diferencia no está en el código: está en que el sistema operativo lo ha cargado en memoria y lo está ejecutando.

La burbuja de memoria

Cada proceso vive en la suya



1. **Código** — las instrucciones del programa
2. **Estado** — los valores actuales de las variables
3. **Contador de programa** — qué instrucción toca ejecutar ahora
4. **PID** — el carnet de identidad del proceso

ℹ️ Si un proceso se cuelga, los demás no se enteran: cada uno vive en su **burbuja aislada**.

El PID y las características

El carnet de identidad del proceso

mipid.py — Python

```python
import os
print(f"Mi PID es {os.getpid()}")
```

Ver los procesos del sistema

- **Windows**: Administrador de tareas → "Detalles", o `tasklist`
- **Linux / macOS**: `ps aux` o `top`

Propiedad	Descripción
PID	Identificador único numérico
Memoria propia	Espacio de direcciones aislado
Recursos	Archivos, sockets, manejadores
Contexto	Estado de la CPU, registros, contador
Comunicación	Necesita mecanismos externos (pipes, sockets, archivos)

⚠️ La última fila es la clave: los procesos **no comparten memoria por defecto**.

La analogía de la cocina

Un proceso es una receta en marcha

- 📖 La receta escrita en el libro es el **programa**: código muerto en el disco
- 🍳 El cocinero la coge, pone los ingredientes en **su mesa** (memoria) y empieza por el paso 1 (contador de programa)
- 🎫 Cada cocinero recibe su propio **número de pedido** (PID)

✅ Cada cocinero con su receta y su mesa: si uno se quema, los demás siguen cocinando sin enterarse. Eso es el **aislamiento** de la burbuja de memoria.

ℹ️ Más adelante verás qué pasa cuando hay varias cocinas (varias CPUs) o varios restaurantes (varias máquinas).

Mini-chequeo: qué es un proceso

🧠 Responde en 30 segundos:

1. ¿Qué es el PID y para qué sirve?
2. ¿Qué contiene la burbuja de memoria de un proceso?
3. ¿Por qué dos procesos no pueden compartir una variable directamente?

🔄 **Respuestas:**

- El **identificador único numérico** que el SO asigna a cada proceso para referirse a él
- El **código**, el **estado** de las variables, el **contador de programa** y el **PID**
- Porque cada uno vive en su **burbuja aislada**; necesitan mecanismos externos (pipes, sockets, archivos)

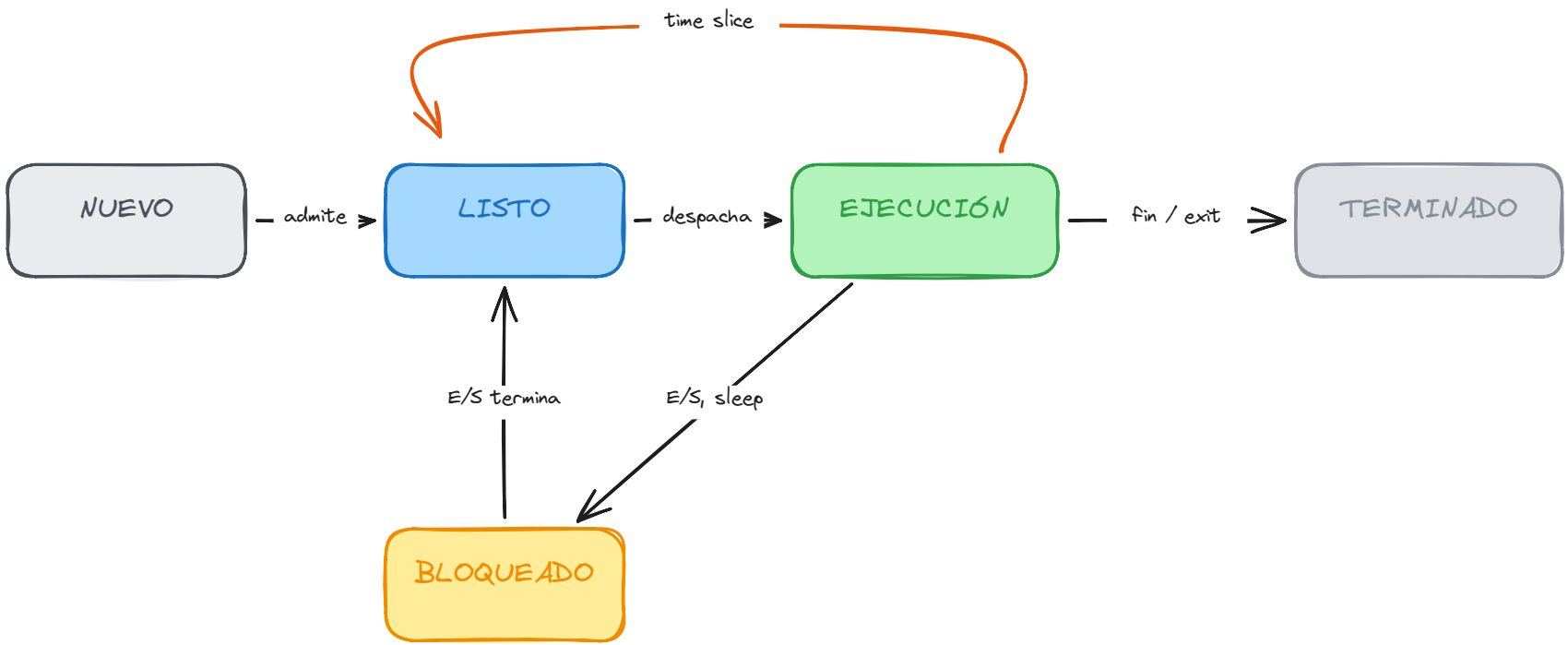
Estados de un proceso

La idea en una frase

> Un proceso no está siempre "ejecutándose": nace, espera su turno, usa la CPU, se bloquea esperando recursos y finalmente muere. Esos **cinco estados** forman su ciclo de vida.

El sistema operativo gestiona decenas o cientos de procesos a la vez con una sola CPU (o pocas). Para que todo parezca simultáneo, **reparte la CPU en porciones de tiempo** y cada proceso va pasando de estado según lo que le toca.

El ciclo de vida: las cinco estados



Las transiciones, una a una

Estado	Qué significa
NUEVO	El proceso acaba de ser creado
LISTO	Preparado para ejecutar, esperando que la CPU esté libre
EJECUCIÓN	La CPU está ejecutando sus instrucciones
BLOQUEADO	Esperando un recurso (I/O, socket, sleep)
TERMINADO	El proceso ha finalizado

ℹ️ De **LISTO** solo se puede ir a **EJECUCIÓN** (la CPU toca) y volver · De **BLOQUEADO** se vuelve siempre a **LISTO** cuando la E/S termina.

La cafetería: los estados en acción

Un solo camarero (CPU), muchos clientes (procesos)

1. Entras por la puerta → **NUEVO**
2. Te pones en la cola → **LISTO** (esperando al camarero)
3. El camarero te atiende → **EJECUCIÓN**
4. Pides café y el camarero se va a la máquina → **BLOQUEADO**
5. El café está listo y vuelves a la cola → **LISTO**
6. El camarero te lo entrega → **EJECUCIÓN**
7. Pagas y te vas → **TERMINADO**

ℹ️ El camarero alterna entre los clientes dando a cada uno unos segundos: por eso una sola CPU atiende a muchos procesos "a la vez".

Mini-chequeo: estados

🧠 Responde sin mirar:

1. ¿Qué diferencia hay entre LISTO y BLOQUEADO?
2. ¿Qué transición ocurre cuando un proceso llama a `time.sleep(2)`?
3. Un proceso está ejecutándose y el planificador le quita la CPU por prioridad. ¿A qué estado pasa?

🔄 **Respuestas:**

- LISTO = "solo espero a que la CPU me toque" · BLOQUEADO = "no puedo ejecutar aunque me dieran la CPU: espero un recurso"
- **EJECUCIÓN** → **BLOQUEADO**: pide una espera de E/S
- A **LISTO**: sigue preparado, solo perdió su turno

Paralela vs Distribuida

La idea en una frase

> **Paralela** es ejecutar varias tareas **a la vez** en varias CPUs; **distribuida** es ejecutarlas en **múltiples máquinas** conectadas por red; **concurrencia** es que varias tareas **avancen**, aunque se turnen en una sola CPU.

Concepto	Qué significa
Paralela	Varias tareas a la vez en múltiples CPUs/núcleos
Distribuida	Varias tareas en múltiples máquinas por red
Concurrencia	Varias tareas avanzando (pueden turnarse)

- 🍳 **Paralela**: 4 sartenes, cada una friendo un huevo (4 CPUs)
- 🏙️ **Distribuida**: 4 restaurantes en 4 ciudades, mismo menú
- 👨‍🍳 **Concurrente**: 1 cocinero alternando 3 pedidos

Paralela en Python: multiprocessing.Pool

Cuatro sartenes de verdad

paralelo.py — Python

```python
# Paralela con multiprocessing (varias CPUs)
from multiprocessing import Pool
def cuadrado(n):
    return n * n
if __name__ == "__main__":
    # obligatorio en Windows: protege el spawn
    with Pool(4) as p:  # 4 procesos en paralelo
        print(p.map(cuadrado, [1, 2, 3, 4]))
# Salida: [1, 4, 9, 16]
```

1. `Pool(4)` crea **4 procesos reales** que el SO reparte entre los núcleos
2. `p.map(funcion, lista)` **reparte la lista** entre ellos
3. Si tu máquina tiene 4 núcleos, los 4 se ejecutan **a la vez**

ℹ️ El **GIL** de Python limita los hilos; con **procesos** (multiprocessing) no existe ese límite: cada proceso tiene su propio intérprete.

Paralela vs Distribuida: el duelo

Pregunta	Paralela	Distribuida
¿Dónde se ejecuta?	Múltiples CPUs de una máquina	Múltiples máquinas por red
¿Comparten memoria?	No por defecto (burbujas aisladas)	No (cada máquina tiene la suya)
¿Comunicación?	Pipes, memoria compartida, locks	Sockets, HTTP, mensajería
¿Escala?	Hasta los núcleos de tu CPU	Hasta cientos de máquinas
Ejemplo en Python	`multiprocessing.Pool`	socket, APIs REST

✅ La paralela es **interna a la máquina**; la distribuida **necesita red** y comunicación por sockets/HTTP.

Mini-chequeo: paralela vs distribuida

🧠 Tres preguntas rápidas:

1. ¿Cuál es la diferencia clave entre paralela y distribuida?
2. ¿Por qué el "cocinero alternando" es concurrencia y no paralelismo?
3. ¿Qué función de `multiprocessing` reparte una tarea entre varios procesos?

🔄 **Respuestas:**

- Paralela = **varias CPUs de una misma máquina** · Distribuida = **múltiples máquinas por red**
- Porque hay **una sola CPU/sartén**: las tareas avanzan turnándose, no a la vez
- `Pool(4).map(funcion, lista)` : crea 4 procesos y reparte la lista

subprocess.run()

La idea en una frase

> `subprocess.run()` lanza un programa y **espera** a que termine, devolviéndote su salida y su código de retorno. Es la forma más sencilla de ejecutar otro programa desde Python.

Cuando necesitas el resultado de un comando (¿qué versión hay? ¿responde el ping? ¿qué dice este comando?), **run() es tu herramienta**: lanza, espera y captura.

El primer lanzamiento con run()

primer_run.py — Python

```python
import subprocess
# Comando simple
resultado = subprocess.run(
    ["python", "--version"],
    capture_output=True,
    text=True
)
print(f"Salida: {resultado.stdout}")
print(f"Código: {resultado.returncode}")
# Salida: Python 3.11.4
# Código: 0
```

1. Una **lista** con el ejecutable y sus argumentos: cada elemento es un argumento aparte
2. `capture_output=True` captura **stdout y stderr**
3. `text=True` devuelve **cadenas**, no bytes
4. `returncode`: **0 = todo bien**, distinto de 0 = algo falló

Parámetros y resultado de run()

Parámetros importantes

Parámetro	Qué hace
`capture_output=True`	Captura stdout y stderr
`text=True`	Devuelve strings en vez de bytes
`timeout=N`	Excepción si tarda más de N segundos
`check=True`	Excepción si returncode ≠ 0

El objeto resultado

- `resultado.stdout` — la salida estándar
- `resultado.stderr` — la salida de errores
- `resultado.returncode` — 0 si fue bien

⏱️ Timeout: no esperes para siempre

timeout.py — Python

```python
try:
    r = subprocess.run(
        ["ping", "google.com", "-n", "3"],
        capture_output=True, text=True,
        timeout=10)
    print(r.stdout)
except subprocess.TimeoutExpired:
    print("❌ El ping tardó demasiado")
except subprocess.CalledProcessError:
    print("❌ Error en el comando")
```

Un ejemplo con error controlado

run() por defecto no lanza excepciones: devuelve

error.py — Python

```python
import subprocess
resultado = subprocess.run(
    ["python", "--versio"],
    capture_output=True, text=True)
print(f"Código: {resultado.returncode}")
print(f"Error: {resultado.stderr}")
```

```
$ python error.py
Código: 2
Error: Unknown option: --versio
```

ℹ️ El código de retorno **2** (distinto de 0) te dice que el comando falló, y **stderr** te dice por qué. Sin excepciones: por defecto `run()` solo devuelve el resultado.

Mini-chequeo: subprocess.run()

🧠 Comprueba lo aprendido:

1. ¿Qué hacen `capture_output=True` y `text=True`?
2. ¿Qué significa un returncode distinto de 0?
3. ¿Qué excepción lanza `run()` si el comando tarda más de `timeout`?

🔄 **Respuestas:**

- `capture_output` captura stdout y stderr; `text=True` los devuelve como **strings**
- Que el comando **falló** (0 = éxito, otro número = error)
- `subprocess.TimeoutExpired`

subprocess.Popen()

La idea en una frase

> `subprocess.Popen()` lanza un proceso en **segundo plano** y devuelve el control inmediatamente: no espera a que termine. Tú sigues haciendo cosas mientras él vive.

⚠️ **run() espera; Popen() no.** Eso convierte a Popen en la herramienta para abrir aplicaciones, lanzar varios programas a la vez y gestionarlos después con sus métodos.

El primer Popen: abrir el bloc de notas

popen.py — Python

```python
import subprocess
# Lanzar el bloc de notas (no espera)
proceso = subprocess.Popen(["notepad.exe"])
print(f"Lanzado con PID {proceso.pid}")
# Python sigue mientras el bloc está abierto
print("Haciendo otras cosas...")
# Cuando queramos, esperamos a que termine
proceso.wait()
print("El bloc de notas se cerró")
```

1. `Popen([...])` lanza el bloc y **vuelve al instante**
2. `proceso.pid` da el **PID** del proceso recién creado
3. Python **sigue ejecutando** mientras el bloc está abierto
4. `proceso.wait()` **bloquea** hasta que se cierra

Los métodos de Popen

Método	Qué hace
`proceso.wait()`	Espera a que termine (bloqueante)
`proceso.poll()`	Pregunta si ha terminado (no bloqueante)
`proceso.terminate()`	Envía señal de terminación
`proceso.kill()`	Mata el proceso forzosamente
`proceso.pid`	PID del proceso hijo

ℹ️ **wait() vs poll()**: wait() **bloquea** hasta que termina; poll() **no bloquea**: devuelve `None` si sigue vivo, o su código de retorno si ya terminó.

poll.py — Python

```python
import subprocess, time
proceso = subprocess.Popen(["notepad.exe"])
while proceso.poll() is None:
    print("El bloc sigue abierto...")
    time.sleep(1)
print(f"Cerrado con código {proceso.poll()}")
```

terminate() vs kill(): el cierre suave y el mazazo

🤝 **terminate()**: en Linux/macOS envía **SIGTERM**, un cierre *suave*: el proceso puede guardar y salir

🔨 **kill()**: mata **forzosamente**, sin dar opción. Úsalo solo cuando terminate() no baste

⚠️ En **Windows** ambos son equivalentes: terminate() llama a TerminateProcess, una terminación **forzosa** igual que kill().

cierra.py — Python

```python
import subprocess
proceso = subprocess.Popen(["calc.exe"])
print(f"Calculadora con PID {proceso.pid}")
proceso.terminate()  # cierre suave (Linux) o forzoso (Windows)
proceso.kill()       # solo si el anterior no funcionó
```

Lanzar varios procesos a la vez

La gran ventaja de Popen

varios.py — Python

```
import subprocess, time
notepad = subprocess.Popen(["notepad.exe"])
calc = subprocess.Popen(["calc.exe"])
mspaint = subprocess.Popen(["mspaint.exe"])
print(f"PIDs: {notepad.pid}, {calc.pid}, {mspaint.pid}")
time.sleep(5)  # les damos 5 segundos de vida
notepad.terminate()
calc.terminate()
mspaint.kill()
print("Todos cerrados 🏁")
```

✅ Los 3 procesos viven **a la vez**, cada uno con su PID, mientras Python hace otras cosas. Eso es imposible con `run()`, que espera a cada uno antes de lanzar el siguiente.

Mini-chequeo: Popen

🧠 Las tres claves del segundo plano:

1. ¿Qué diferencia hay entre `wait()` y `poll()`?
2. ¿Qué devuelve `proceso.poll()` mientras el proceso sigue vivo?
3. ¿Cuándo usarías `kill()` en lugar de `terminate()`?

🔄 **Respuestas:**

- `wait()` **bloquea** hasta que termina; `poll()` **pregunta** sin bloquear
- `None` (el proceso sigue ejecutándose)
- Cuando terminate() no consigue cerrarlo: kill() mata **forzosamente**

multiprocessing: procesos nativos

La idea en una frase

> Mientras que `subprocess` lanza **programas externos** (la calculadora, un comando del sistema), `multiprocessing` **clona tu propio programa** Python para ejecutar varias de tus funciones al mismo tiempo en distintos núcleos del procesador.

Característica	subprocess	multiprocessing
¿Qué lanza?	Ejecutables externos (ping, notepad.exe, ls)	Funciones de tu propio código Python
Paso de datos	Difícil: tuberías de texto (stdin/stdout)	Fácil: variables y objetos como argumentos
Uso típico	Automatización del SO y scripts	Concurrencia, servidores y cálculos pesados

ℹ️ ¿Tu propio servidor debe atender a **cinco clientes a la vez**? Con `subprocess.Popen` no puedes hacerlo de forma elegante para cada tarea interna. Aquí entra la librería estándar **multiprocessing**.

✅ La base es la clase **Process**: le dices qué **función** (target) y qué **argumentos** (args) debe ejecutar un proceso clon de tu programa.

Process, start() y join()

Clona tu propio programa

trabajador.py — Python

```python
import multiprocessing, os, time
def trabajador(nombre, repeticiones):
    print(f"[{os.getpid()}] Soy el trabajador {nombre}")
    time.sleep(repeticiones)
    print(f"[{os.getpid()}] {nombre} ha terminado")
if __name__ == "__main__":
    # IMPORTANTE en Windows: protege el main
    print(f"[{os.getpid()}] Soy el proceso PADRE")
    p1 = multiprocessing.Process(
        target=trabajador, args=("Alice", 2))
    p2 = multiprocessing.Process(
        target=trabajador, args=("Bob", 1))
    p1.start()  # pasan al estado LISTO/EJECUCIÓN
    p2.start()
    p1.join()   # espera bloqueante, como wait()
    p2.join()
    print("Todos los trabajadores han finalizado")
```

1. `target`: el nombre de la función **sin paréntesis** (no la ejecutas ahora, se la pasas al proceso)
2. `args`: tupla con los argumentos; con uno solo, **recuerda la coma final**: `("Alice",)`
3. `start()`: lanza el proceso al **planificador** del SO
4. `join()`: bloquea al padre hasta que **este hijo** termine

⚠️ `if __name__ == "__main__"` es **obligatorio en Windows**: sin él los hijos reimportan el script y entran en un **bucle infinito de procesos** (lo veremos en el bloque 02).

Resumen — Bloque 01 completado

Lo que hemos aprendido

📦 Sección 1

Qué es un proceso
Burbuja de memoria, PID, aislamiento, analogía de la cocina

🔄 Sección 2

Estados
NUEVO, LISTO, EJECUCIÓN, BLOQUEADO, TERMINADO y el planificador

🤹 Sección 3

Paralela vs distribuida
CPUs vs máquinas, concurrencia, multiprocessing.Pool

🚀🧬 Secciones 4-5 · multiprocessing

subprocess y procesos nativos
run() para esperar y capturar · Popen() para segundo plano · Process con start() y join()

✅ **Ya sabes qué es un proceso, cómo vive y cómo lanzarlo desde Python.** En el bloque 02: comunicación por pipes, compartir memoria con Manager, sincronizar con Lock, Windows vs Linux y práctica real. 🚀

// EDUCACIÓN A DISTANCIA // EDUCACIÓN PÚBLICA Y GRATUITA // EDUCACIÓN DE CALIDAD // EDUCACIÓN AUTONÓMICA //